

INSTRUÇÕES

- I. A prova é em dupla, **offline** e com consulta apenas ao material da dupla. Não é permitido uso de celular, se caso for ao banheiro, deixe o celular na mesa do professor.
- II. Leiam a prova com atenção e em silêncio. A avaliação tem 6 questões, cada uma tem seu valor especificado antes do enunciado.
- III. Ao término da prova, **chame o professor para recolher seus arquivos. Coloque seus arquivos em uma pasta, compacte, e entregue um arquivo zip. O nome do arquivo zip deverá ser o nome completo da dupla.**
- IV. A prova se inicia às 09h30m e terminará às 13h00m. Não serão aceitas entregas tardias. Tempo mínimo de permanência: 50m.
- V. Dúvidas? Retorne ao primeiro item.

CENÁRIO

Você foi contratado por uma empresa de consultoria financeira para analisar dados da Receita Federal. Você recebeu um arquivo CSV bruto (socios.csv) de aproximadamente 2 GB contendo dados de sócios de empresas. Sua missão é preparar esse ambiente, importar os dados com eficiência, otimizar consultas e garantir a segurança (backup) das informações.

QUESTÕES

- 1)** (1,0 Ponto) O arquivo recebido original está codificado em ISO-8859-1 (padrão antigo), mas o MongoDB trabalha nativamente com UTF-8.
- a) Converta o arquivo socios.csv para a codificação UTF-8.
 - b) Verifique se o delimitador é vírgula (,) ou ponto e vírgula (;) e ajuste se necessário.
 - c) Evidência: Print da ferramenta ou comando utilizado para a conversão e verificação.

Instruções: para executar os scripts Python, certifique-se que tanto o CSV quanto o script estão na mesma pasta. Pelo terminal, execute usando: **python preparar_dados.py** (veja instruções detalhadas abaixo)

- 2)** (3,0 Pontos) Realize a importação dos dados para o MongoDB, sendo Banco de Dados: **receita_federal** e Coleção: **socios**. Requisito Técnico: Como o arquivo é grande (2 GB) e o tempo é curto, você deve utilizar o parâmetro de paralelismo (**numInsertionWorkers=2**) conforme visto em aula para acelerar o processo. Utilize a primeira linha do arquivo como cabeçalho.
- a) Evidência 1: Print do comando completo **mongoimport** executado.
 - b) Evidência 2: Print do terminal mostrando a barra de progresso e a mensagem final de conclusão com o número de documentos importados.
- 3)** (1,0 Ponto) Durante a execução da importação (Questão 2), abra o Gerenciador de Tarefas (ou monitor do sistema) e analise o consumo de hardware. Print mostrando o uso de CPU e Disco pelo processo do MongoDB ou do terminal, comprovando (ou não) o impacto do uso de múltiplas threads.

- 4) (2,0 Pontos) Após a importação, note que as consultas por **nome_socio** estão lentas. Crie um índice simples para o campo que representa o nome do sócio. Faça uma consulta (**find**) buscando todos os sócios que contenham o mesmo nome que o seu, limitando o resultado a 5 documentos. Print do comando de criação do índice e print do resultado da consulta.
- 5) (1,5 Ponto) O departamento de marketing solicitou um arquivo JSON contendo apenas sócios que entraram na sociedade a partir de 01/01/2025. Utilize o **mongoexport** para gerar um arquivo chamado **marketing_novos_socios.json**. Use uma query para filtrar apenas a cidade desejada e print do comando **mongoexport** utilizado.
- 6) (1,5 Ponto) Para finalizar o serviço, você deve garantir que os dados importados estejam salvos em formato binário (BSON) para uma eventual restauração completa.
- a) Utilize o **mongodump** para fazer o backup apenas do banco **receita_federal**. Salve o backup na pasta **backup_prova_final**.
 - b) Utilize o parâmetro de compressão para economizar espaço em disco.
 - c) Print do comando executado e print da pasta criada com os arquivos **.bson** e **.metadata.json** dentro.

PREPARAÇÃO DOS DADOS COM PYTHON

Instruções:

- 1) Crie um arquivo **preparar_dados.py** na mesma pasta onde estão os CSV
- 2) Abra o terminal na pasta e execute: **python preparar_dados.py**
- 3) Siga as instruções do script

```
import csv
import os
from tqdm import tqdm

def formatar_csv(arquivo_entrada, arquivo_saida, delimitador_entrada=';',
delimitador_saida=','):
    print("Contando o total de linhas do arquivo (isso pode levar alguns segundos em
arquivos gigantes)...")

    # Conta as linhas para a barra de progresso
    with open(arquivo_entrada, 'r', encoding='utf-8') as f:
        total_linhas = sum(1 for _ in f)

    print(f"Total de {total_linhas} linhas encontradas. Iniciando processamento...")

    # Abre os arquivos de leitura e gravação
```

```
with open(arquivo_entrada, 'r', encoding='utf-8') as f_in, \
    open(arquivo_saida, 'w', encoding='utf-8', newline='') as f_out:

    # LEITOR: Configurado para ler com o delimitador original (ex: ponto e vírgula
';')
    leitor = csv.reader(f_in, delimiter=delimitador_entrada)

    # ESCRITORES: Configurados para gravar com o NOVO delimitador (ex: vírgula ',')
    # 1. Cabeçalho (sem aspas)
    escritor_cabecalho = csv.writer(f_out, delimiter=delimitador_saida,
quoting=csv.QUOTE_NONE, escapechar='\\')
    # 2. Dados (com aspas em todos os campos)
    escritor_dados = csv.writer(f_out, delimiter=delimitador_saida,
quoting=csv.QUOTE_ALL)

    # Processamento linha a linha com barra de progresso
    for indice, linha in tqdm(enumerate(leitor), total=total_linhas,
desc="Processando CSV", unit="linhas"):
        if indice == 0:
            escritor_cabecalho.writerow(linha)
        else:
            escritor_dados.writerow(linha)

    print("\nProcesso concluído com sucesso!")

# =====
# Exemplo de uso:
# =====
if __name__ == "__main__":

    caminho = input("Digite o caminho do arquivo CSV: ").strip()
    nome, ext = os.path.splitext(caminho)
    novo_arquivo = f"{nome}_com_aspas{ext}"

    tamanho_total = os.path.getsize(caminho)

    nome_completo = nome + ext

    # Executa a função definindo o que entra (;) e o que sai (,)
    formatar_csv(
        arquivo_entrada=nome_completo,
        arquivo_saida=novo_arquivo,
        delimitador_entrada=';',
        delimitador_saida=',',
    )
```